VIGILANT: An Auditable Evaluation of Change-Aware Software Traceability and Evidence-Based Investigation

Technical research draft for author and supervisor review
Author, affiliation and publication venue not yet supplied

***Abstract*—Software changes can invalidate relationships between requirements, implementations, tests and API clients. VIGILANT integrates versioned artifact extraction, lexical and embedding retrieval, a structural knowledge graph, bounded investigation tools and evidence-checked model responses. We report an executed evaluation rather than a proposed architecture alone. The preserved study contains 251 local-model traceability reranking calls over four linked-query populations, 60 consistency-ablation calls, twelve controlled Git-change scenarios and an end-to-end API-change investigation. Independent arithmetic recomputation checks 23 retrieval-method rows and 1484 per-query metric rows. Results are mixed: graph context raises eTour mean average precision from 0.4406 to 0.4744 but reduces fixed-cutoff F1 from 0.3804 to 0.3713. The frozen RAG reranker slightly improves mean average precision on two populations and reduces it on two. Every local-model consistency variant returns uncertainty in every case, matching the 25% authored-label agreement of always-uncertain prediction. These findings support the feasibility of an inspectable, recoverable investigation pipeline, but do not establish superior consistency classification or independently verified semantic grounding.**

***Index Terms*—software traceability, software evolution, retrieval-augmented generation, knowledge graph, reproducibility**

I. Introduction

A backend API can change while a generated client retains the earlier route. A useful maintenance tool should connect the changed code to possible consumers, locate requirements and tests, and show the source versions behind each finding. Similarity rankings alone do not establish a defect. A language-model explanation is also insufficient when a reviewer cannot recover its cited evidence.

VIGILANT addresses this engineering problem through a version-aware investigation pipeline. Git commits become canonical artifact snapshots; retrieval proposes trace candidates; parsed relationships provide graph context; and a bounded tool workflow gathers and verifies evidence before storing rule findings and a separate model decision. This separation prevents a plausible dependency from silently becoming a confirmed implementation link or consistency violation.

The contribution is an implemented experimental artifact, its reproducible controlled-change benchmark and a candid evaluation of the resulting behavior. The study asks: (RQ1) how do lexical, embedding, hybrid, graph and RAG methods compare on the selected linked-query datasets; (RQ2) do additional context and investigation tools improve a fixed local model's agreement with consistency labels; and (RQ3) which execution and evidence properties are established by the recorded pipeline? We distinguish these questions from an unmeasured claim of general superiority.

II. Related Work and Positioning

BM25 provides a probabilistic retrieval foundation [1]; reciprocal rank fusion combines ranked lists [2]; and Sentence-BERT supports separately encoded sentence representations [3]. Our lexical and embedding arms use these ideas as implemented baselines rather than substituting published scores for our own runs.

RAG combines parametric and retrieved information [4]. LiSSA applies retrieval-augmented generation to generic software traceability [5]. TransArC relates documentation and code through an intermediate architecture model [6], while related ARDoCo work investigates architectural documentation inconsistencies through trace recovery [7]. These precedents preclude a claim that VIGILANT is the first LLM-based traceability system.

GraphRAG uses an entity graph and community summaries for corpus-level questions [8]. VIGILANT instead uses parsed software relationships and bounded local traversal. SWE-agent investigates agent-computer interfaces for software engineering [9]; VIGILANT's current tool order is scripted. Neither external benchmark scores nor autonomous-agent capabilities are claimed as reproduced here. The literature selection is targeted; comprehensive full-text synthesis remains future work.

III. System and Evidence Contract

A. Artifacts, versions and graphs

An artifact identity incorporates project, repository, path, type and symbol. A separate content hash, commit and line range preserve the source version. Parsers extract supported Python, Java, JavaScript and TypeScript artifacts, requirements, tests, APIs and frontend calls. Static-analysis ambiguities and parse failures remain limitations rather than disappearing into a perfect-extraction claim.

The directed multigraph stores typed relationships including calls, imports, tests, API exposure/consumption, dependencies and commit modifications. Unique static matches are preferred; dynamic or ambiguous calls can remain unresolved. A change investigation combines baseline and changed graphs and performs bounded traversal, normally two hops. Broad

TABLE I
EVALUATED POPULATIONS. RTMX* IS THE MODULE-VALID SUBSET.

Dataset	Req.	Linked	Targets	Links
SMOS	67	67	100	1044
iTrust	131	105	226	286
eTour	58	57	116	308
RTMX*	22	22	7	22

reverse-containment propagation is excluded to limit irrelevant expansion. Similarity-derived trace candidates retain review-required status rather than being promoted to verified `IMPLEMENTS` edges.

B. Retrieval and RAG

TF-IDF uses sublinear term frequency. The pinned MiniLM model [10] encodes up to eight chunks of 180 words per artifact, averages normalized chunk embeddings, then normalizes the aggregate. Truncation can exclude relevant code. Hybrid retrieval combines BM25 and embedding ranks with

$$S(d,q) = \frac{1}{60 + r_{\mathrm{BM25}}(d,q)} + \frac{1}{60 + r_{\mathrm{emb}}(d,q)}. \qquad (1)$$

The Java graph experiment uses normalized hybrid relevance with weight 0.85 and mean adjacent-file relevance with weight 0.15. It uses parsed calls, imports and tests without gold trace edges. This fixed weight is exploratory, not a preregistered confirmatory choice.

The frozen RAG protocol supplies a requirement prefix of at most 900 characters and five candidate prefixes of at most 1100 characters each. Qwen2.5-1.5B-Instruct [11] labels each candidate supported, unsupported or uncertain. Supported candidates move to the front; all others retain their relative order. The rest of the full target ranking remains unchanged. Consequently, reranking cannot improve recall at cutoffs five or ten. This restriction is central to interpreting the experiment.

C. Investigation and verification

The recorded investigation resolves commits, imports snapshots, detects changes, constructs two graphs, investigates impact, recovers trace candidates, retrieves evidence, applies conservative rules, verifies quotes, calls the model and persists a report. Its named roles organize tool use but do not imply autonomous planning. Evidence retains exact text, artifact identity, commit, path, line range and hash. The verifier checks source equality and model citations against supplied identifiers.

Consistency responses require a label, reason, uncertainty, affected identifiers, citations and bounded confidence. Invalid generated responses are retained and handled conservatively. A runtime failure without generated output is not scored as a model prediction. Source fidelity and schema validity are measurable properties; neither proves that evidence entails the conclusion. Rule findings and model decisions remain separate in storage and display.

IV. EXPERIMENTAL SETUP

A. Populations and ground truth

Table I identifies the evaluated populations. Only requirements with resolved gold links enter ranking metrics, excluding 26 iTrust requirements and one eTour requirement. The same policy is used across methods. It does not evaluate rejection of unlinked requirements. SMOS, iTrust and eTour originate from a pinned LiSSA replication snapshot. Train/validation/test roles apply to learned-ranker selection. RTMX is a separate module-level subset: missing modules and unresolved function labels prevent a comparable 24-function evaluation.

VIGILANT-ChangeBench contains twelve real controlled Git mutations: requirement change/addition, code change/deletion, test change/deletion, frontend change, API change, backend change, consumer change and two forms of test drift. Baseline traces are seeded where required. Labels and affected sets are authored and incomplete; these are regression contracts rather than independently annotated natural changes. SAFA is evaluated separately as versioned artifact-tree comparison.

B. Model, metrics and reproducibility

The local model runs on CPU with four inference threads, bfloat16 weights, greedy decoding, a 3072-token input limit and at most 220 generated tokens. Exact model revisions are stored in `data/models.json`. Model weights and dependencies are restored from pinned manifests. Prompt, evidence, schema, provider source, model settings and prepared data must match before a generated observation is reused.

Precision, recall and F1 at ten are micro-averaged over predicted links. Macro recall averages per-query recall. Mean reciprocal rank and mean average precision (MAP) use the full target ranking. With binary relevance, average precision for query q is

$$AP(q) = \frac{1}{|G_q|} \sum_{k=1}^{|D|} P_q(k)\, \mathbf{1}[d_k \in G_q]. \qquad (2)$$

We also preserve nDCG@10 and recall at one, five and ten. Full raw rankings and unrounded metrics remain available. Independent audit scripts recomputed 23 method rows and 1484 query rows, and validated all 251 actual RAG generations against their provenance and schema. This is independent arithmetic verification, not independent human labeling.

The consistency ablation evaluates the same twelve cases in five configurations: E, LLM-only; F, hybrid RAG; G, RAG plus graph context; H, additional bounded investigation; and I, additional temporal context and rules. The experiment changes several context factors and cannot isolate each mechanism's causal effect. Reused observations retain their original timings and cold-loading effects.

V. RESULTS

A. Traceability

Table II reports MAP for all completed retrieval variants. No method dominates every population. BM25 is strongest

TABLE II
MAP FROM COMPLETE SAVED RANKINGS; – MEANS NOT RUN. RTMX* USES SEVEN MODULES.

Dataset	TF-IDF	BM25	MiniLM	Hybrid	+KG	+RAG
SMOS	0.2924	0.2979	0.2859	0.3026	0.2965	0.3061
iTrust	0.3160	0.3787	0.2061	0.3068	0.2958	0.3043
eTour	0.4102	0.3074	0.4492	0.4406	0.4744	0.4427
RTMX*	0.7894	0.7606	0.6795	0.7515	–	0.6985

TABLE III
FROZEN RAG: PAIRED MAP DIFFERENCES RELATIVE TO HYBRID.

Dataset	Calls	MAP difference	Mean seconds
SMOS	67	+0.0035	70.82
iTrust	105	-0.0025	42.23
eTour	57	+0.0021	37.93
RTMX*	22	-0.0530	67.28

on iTrust at 0.3787. MiniLM exceeds hybrid on eTour MAP, while hybrid has a higher micro F1@10. The graph variant raises eTour MAP to 0.4744 from 0.4406 but lowers micro F1@10 to 0.3713 from 0.3804. Graph results on SMOS and iTrust are lower than their hybrid MAP values. RTMX graph reranking was not run under the Java-file protocol.

The learned ranker uses TF-IDF, normalized BM25, embedding and token-Jaccard features. It was trained on SMOS and rejected because its iTrust validation MAP, 0.3051, did not exceed BM25's 0.3787. eTour test results were not used to retain it. Preserving this rejection avoids selecting a favorable test result after inspection.

B. RAG reranking

The study completed 67 SMOS, 105 iTrust, 57 eTour and 22 RTMX calls. Table III reports the paired MAP differences. Gains on SMOS and eTour are small; iTrust and RTMX decline. On eTour, MRR and macro recall at one decline despite slightly higher MAP and nDCG. No statistical significance or general superiority claim follows. All 251 outputs pass the independent full-schema check, which is not a semantic-grounding rate.

RTMX's seven target modules make recall at ten equal to one regardless of ranking. A larger-looking MAP there cannot be interpreted as evidence that RTMX is easier in an equivalent evaluation setting. Candidate-prefix truncation and the top-five selection policy limit what the model can assess. The reranker's order changes leave recall at five and ten fixed by construction.

C. Consistency ablation and change analysis

Every ablation variant returns UNCERTAIN for every case. Table IV shows 25% agreement with authored labels, exactly the always-uncertain comparator. Five generated responses fail citation validation and become uncertain. Greater known-endpoint coverage in later variants does not yield better final consistency classification. Citation validity and partial-oracle coverage must remain distinct from semantic accuracy.

TABLE IV
TWELVE CASES PER VARIANT. AGREE.: AUTHORED-LABEL AGREEMENT; UNC.: UNCERTAINTY; CITE.: CITATION VALIDITY; END.: KNOWN-ENDPOINT RECALL.

Variant	Agree.	Unc.	Cite.	End.
E	0.250	1.000	1.000	0.000
F	0.250	1.000	0.917	0.000
G	0.250	1.000	0.917	0.200
H	0.250	1.000	0.917	0.320
I	0.250	1.000	0.833	0.720

All twelve deterministic change scenarios contain their expected status somewhere in the findings. Their partial expected sets contain 18 endpoints: 14 are recovered and 4 missed. Predictions outside those sets are unjudged, not verified false positives. Expected-status presence is not a complete classification-accuracy measure. SAFA comparison finds 205 added, 85 deleted and nine modified nodes between versions of 240 and 360 nodes; its labels do not establish code-consistency accuracy.

D. Executed API-change example

The demonstration changes a backend route prefix from `/items` to `/catalog`; the generated client still requests `/api/v1/items/{id}`. Twelve recorded tool stages complete, yielding 11 changed artifact records, 33 impact candidates and 24 source-verified excerpts. Fifteen rule findings share the route-renaming cause and are not fifteen independent confirmed defects. The model returns UNCERTAIN. Recorded end-to-end time is 33.65 seconds for this run, not a controlled speed benchmark. The preserved commit pair, evidence identifiers and raw report allow this example to be reopened without inventing a live demonstration result.

VI. DISCUSSION AND THREATS TO VALIDITY

RQ1 has a population- and metric-dependent answer. Graph context helps one ranking objective in eTour while reducing another, and the RAG reranker does not expand the retrieved candidate set. RQ2 has a negative result for the tested model and protocol: additional context does not improve observed agreement over always-uncertain prediction. The experiment does not identify whether model capacity, prompt construction, context selection, schema restrictions or label ambiguity explains that result. RQ3 is supported as execution evidence: the pipeline persists versions, tool outcomes and source-verifiable excerpts. It is not evidence of independent semantic reliability.

The datasets are small and heterogeneous. Public benchmark contamination in model pretraining is unknown. Linked-query filtering omits unlinked requirements. Controlled cases are correlated and authored, with seeded traces and partial expected sets. Static extraction can miss reflection, dynamic dispatch and route construction. Graph scores are heuristic rather than calibrated probabilities. Prefix truncation may omit decisive behavior. The consistency pilot uses one small local model and a compound context ablation.

Per-call timings include unequal cold-load histories and resumed observations, so they are unsuitable for a causal speed comparison. API charges were zero for the local calls; compute and energy costs were not measured as zero. Browser interaction and Windows execution remain unverified. Thirty preserved automated tests and a production frontend build do not substitute for those checks. Independent evidence-sufficiency annotation and naturally occurring changes are necessary before claiming robust consistency detection.

VII. Conclusion

VIGILANT is an executed, auditable implementation connecting Git changes, artifacts, structural graphs, retrieval and evidence-backed investigation. Its completed study preserves 251 trace-reranking calls, 60 ablation calls, twelve controlled cases and a full model-enabled investigation. The results constrain the claim: retrieval effects are mixed and the tested local model does not improve consistency classification. Future evaluation should use independently annotated natural changes, separately assessed evidence sufficiency, controlled context ablations and additional model settings selected without test leakage. This draft requires author, institutional and external scientific review before submission.

References

[1] S. Robertson and H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. 2009. DOI: 10.1561/1500000019.

[2] G. V. Cormack, C. L. A. Clarke and S. Buettcher. *Reciprocal rank fusion outperforms condorcet and individual rank learning methods*. SIGIR 2009, pp. 758-759. DOI: 10.1145/1571941.1572114.

[3] N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP 2019, pp. 3982-3992. DOI: 10.18653/v1/D19-1410.

[4] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401, 2020. Primary record.

[5] D. Fuchss, T. Hey, J. Keim, H. Liu, N. Ewald, T. Thirolf and A. Koziolek. *LiSSA: Toward Generic Traceability Link Recovery Through Retrieval-Augmented Generation*. ICSE 2025, pp. 1396-1408. DOI: 10.1109/ICSE55347.2025.00186.

[6] J. Keim, S. Corallo, D. Fuchss, T. Hey, T. Telge and A. Koziolek. *Recovering Trace Links Between Software Documentation And Code*. ICSE 2024, article 215. DOI: 10.1145/3597503.3639130.

[7] J. Keim, S. Corallo, D. Fuchss and A. Koziolek. *Detecting Inconsistencies in Software Architecture Documentation Using Traceability Link Recovery*. ICSA 2023, pp. 141-152. DOI: 10.1109/ICSA56044.2023.00021.

[8] D. Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024; viewed revision v2, 2025. Primary record.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan and O. Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793, 2024. Primary record.

[10] Sentence Transformers. *all-MiniLM-L6-v2 model card*. Model publisher. Accessed 7 October 2026. Experimental revision recorded in `data/models.json`.

[11] Qwen. *Qwen2.5-1.5B-Instruct model card*. Model publisher. Accessed 7 October 2026. Experimental revision recorded in `data/models.json`.